MEDIUM

Secondary Scope — overlay/

TwoStep Broadcast: Deduplication Check Placed After Rebroadcast

Repo: ton-blockchain/ton **Branch:** testnet **Commit:** `3bb6abc` **Scope:** Secondary — overlay/broadcast-twostep.cpp **Date:** 2026-03-14

Summary

In the TwoStep overlay broadcast protocol, `rebroadcast()` is called before the `is_delivered()` deduplication check in both the simple and FEC broadcast code paths. Any node can replay a previously seen broadcast (within the ±20-second timestamp window) and cause every relay node to forward the duplicate to all N−1 peers before recognising it as a duplicate. This enables O(N²) bandwidth amplification per replay.

Vulnerable Code

Simple broadcast path (lines 313–316)

```
// overlay/broadcast-twostep.cpp — process_broadcast(), simple path
if (will_rebroadcast) {
  rebroadcast(overlay, bcast_src_adnl_id, serialize_tl_object(broadcast, true));  // line
314 — relays to N-1 peers
}
if (overlay->is_delivered(broadcast_id)) { return; }  // line 316 — dedup check too late
overlay->register_delivered_broadcast(broadcast_id);
```

FEC broadcast path (lines 346–352)

```
// overlay/broadcast-twostep.cpp — process_broadcast(), FEC path
if (will_rebroadcast) {
  rebroadcast(overlay, bcast_src_adnl_id, serialize_tl_object(broadcast, true));  // line
347 — relays to N-1 peers
}
auto it = broadcasts_.find(broadcast_id);
bool is_new = (it == broadcasts_.end());
if (is_new) {
  if (overlay->is_delivered(broadcast_id)) { return; }  // line 352 — dedup check too late
}
```

Both paths share the same structural problem: the relay decision is made (and executed) before the deduplication check. The `rebroadcast()` implementation iterates all peers unconditionally:

```
// overlay/broadcast-twostep.cpp — rebroadcast()
void BroadcastsTwostep::rebroadcast(OverlayImpl *overlay, ...) {
  overlay->iterate_all_peers([&](const adnl::AdnlNodeIdShort &peer_id, OverlayPeer &) {
    if (peer_id != bcast_src_adnl_id && peer_id != overlay->local_id()) {
```

```
      td::actor::send_closure(overlay->adnl(), &adnl::Adnl::send_message, ...);
    }
  });
}
```

Impact

A single replayed broadcast causes each receiving relay node to immediately forward it to all other peers before checking whether it has already been delivered. For an N-node overlay:

Network size (N)	Relay nodes	Messages per replay	Amplification factor
10	9	72	72×
50	49	2,352	2,352×
100	99	9,702	9,702×
300	299	89,402	89,402×

The timestamp check (`check_date`) accepts messages up to 20 seconds in each direction (a 40-second replay window). Within this window, a Byzantine node can repeatedly replay any broadcast it has previously received, sustaining elevated bandwidth consumption across the overlay with minimal outbound effort.

There is no per-sender deduplication or rate limiting on the relay path, and `check_broadcast()` in `private-overlay.cpp` unconditionally accepts all overlay broadcasts without content-level filtering:

```
// validator/consensus/private-overlay.cpp
void check_broadcast(..., td::Promise<td::Unit> promise) override {
  promise.set_value(td::Unit());   // always accepts — no validation
}
```

Fix

Move the deduplication check before `rebroadcast()` in both paths:

Simple path fix

```
if (overlay->is_delivered(broadcast_id)) { return; }     // check first
overlay->register_delivered_broadcast(broadcast_id);
if (will_rebroadcast) {
  rebroadcast(overlay, bcast_src_adnl_id, ...);           // then relay
}
```

FEC path fix

```
auto it = broadcasts_.find(broadcast_id);
bool is_new = (it == broadcasts_.end());
if (is_new && overlay->is_delivered(broadcast_id)) { return; }   // check before relay
if (will_rebroadcast) {
```

```
  rebroadcast(overlay, bcast_src_adnl_id, ...);
}
```

TON Blockchain Contest Round 2 — Secondary scope finding (overlay/). Submitted by TesseraeVentures.